Task 3 — Web Application Security

Security Testing Report

ApexPlanet Software Pvt. Ltd. — Cybersecurity & Ethical Hacking Internship

Target application: DVWA (Damn Vulnerable Web Application) v1.0.7

Test environment: Kali Linux attacker VM → Metasploitable2 target VM, 192.168.187.4, isolated host-only lab network

All testing was performed in a private, isolated virtual lab against a deliberately vulnerable application. No production systems were targeted.

Scope & Methodology

This report documents the OWASP Top 10 testing performed for Task 3 of the internship (Days 25–36) against DVWA running on a Metasploitable2 target, from a Kali Linux attacker machine, both confined to a host-only lab network. Security level in DVWA was set to "low" unless otherwise noted, to make vulnerable code paths reachable for demonstration.

For every vulnerability class below, this report records: the objective, the exact payload or technique used, evidence that the exploit worked, the fix that should be applied (and, where captured, evidence of that fix), and the risk the issue would represent if it existed in a production system.

Contents: 1) SQL Injection 2) Stored Cross-Site Scripting (XSS) 3) Cross-Site Request Forgery (CSRF) 4) Local File Inclusion (LFI) 5) Burp Suite — Request Tampering & Intruder Fuzzing 6) Web Security Headers (Before / After Fix)

1. SQL Injection

Objective

Confirm that the DVWA "User ID" lookup field is vulnerable to classic and UNION-based SQL injection, and extract credential data directly from the users table to demonstrate real-world impact.

Payloads used

```
1' OR '1'='1

1' UNION SELECT user, password FROM users-- -

1' UNION SELECT table_name, column_name FROM information_schema.columns WHERE table_name='users'-- -
```

Exploit evidence

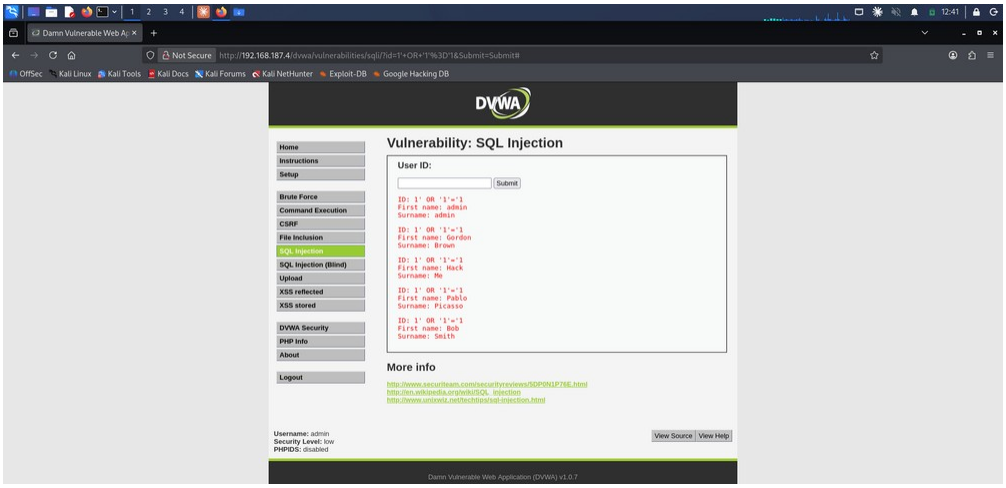


Fig 1.1 — Boolean-based injection (1' OR '1'='1) returns every row in the users table instead of a single user.

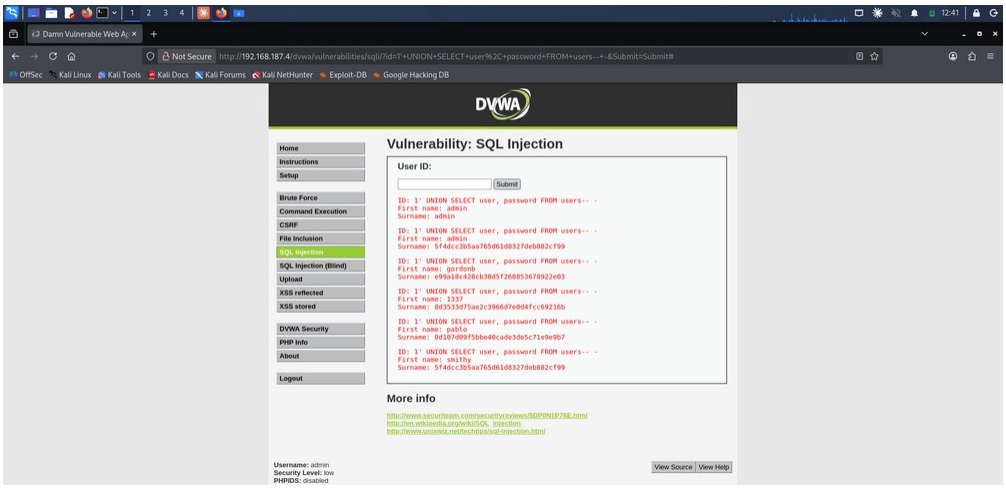


Fig 1.2 — UNION SELECT pulls usernames and MD5 password hashes directly from the users table (e.g. admin, gordonb, 1337, pablo, smithy).

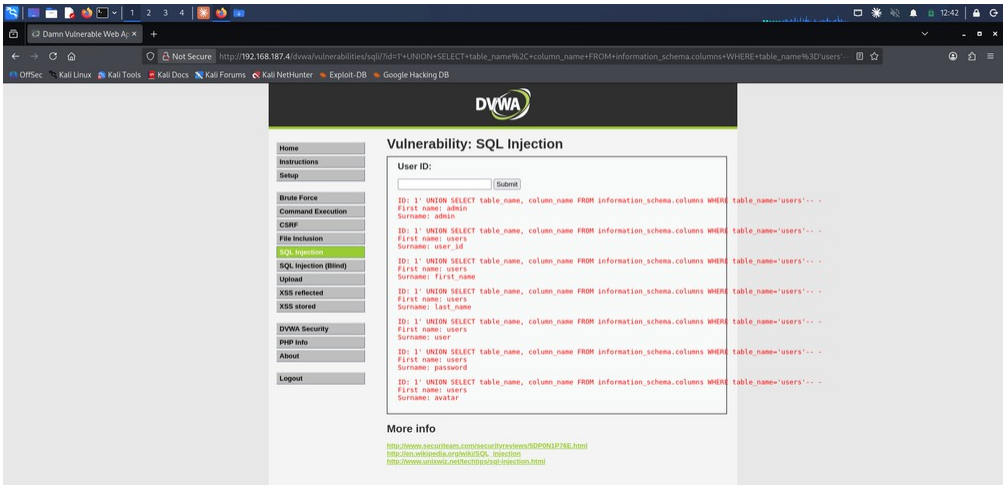


Fig 1.3 — information_schema.columns enumeration reveals the full users table schema (user_id, first_name, last_name, user, password, avatar), which is normally invisible to the application's front end.

Recommended fix: Replace string-concatenated SQL with parameterized queries / prepared statements (PDO bindParam or mysqli bind_param) so user input is never interpreted as SQL syntax. Additionally apply least-privilege DB accounts (the web app user should not be able to read information_schema or other applications' tables) and enable a WAF rule set as defense-in-depth.

Production risk: In production this class of flaw allows full read access to the user table — including password hashes — without authentication, and (depending on DB permissions) can extend to writing/deleting data or pivoting to other databases on the same server. It is rated Critical under OWASP Top 10 (A03:2021 – Injection).

2. Stored Cross-Site Scripting (XSS)

Objective

Prove that the DVWA guestbook ("Name"/"Message" fields) stores attacker-supplied HTML/JavaScript without sanitisation, and that the script executes for every visitor who later views the page — not just the attacker.

Payload used

```
<script>alert('Stored XSS - Ayush')</script>
```

Exploit evidence

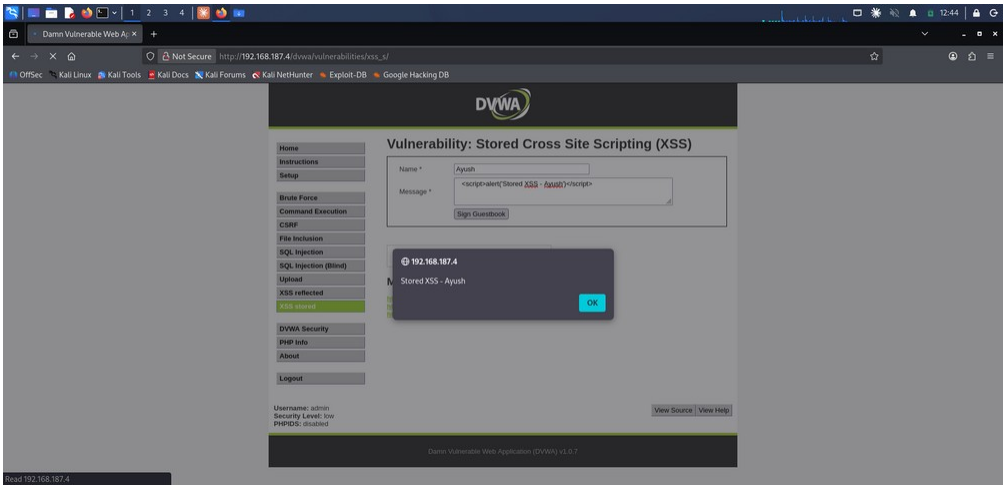


Fig 2.1 — The payload is submitted through the guestbook's Message field and fires immediately, confirming the input is reflected back unescaped.

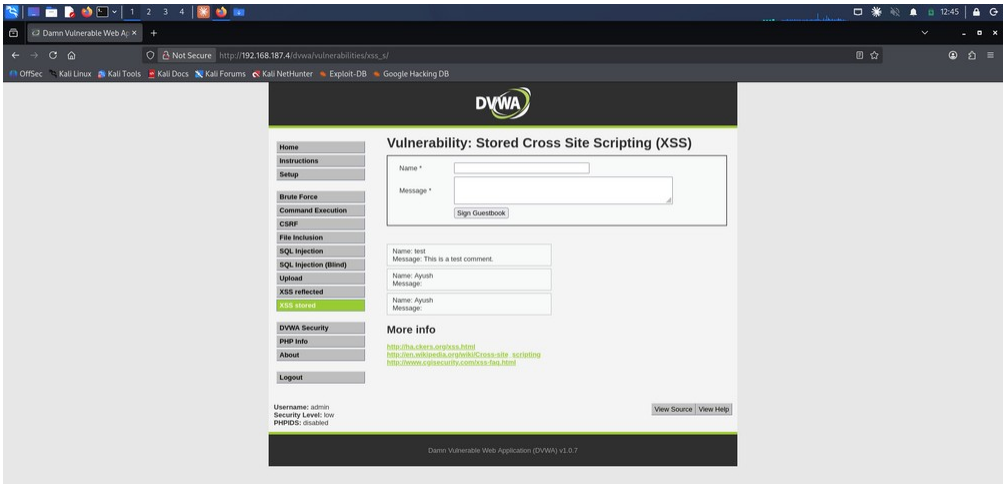


Fig 2.2 — Reloading the guestbook page re-triggers the alert for every visitor (two separate "Ayush" entries persisted with empty message bodies, since the script tag was stripped from display but stored in the DB), proving the payload is stored, not merely reflected.

Recommended fix: Encode all user-supplied output on render (htmlspecialchars()/equivalent templating auto-escape) so "<script>" is displayed as text, not executed. Pair this with strict input validation on write, and add a Content-Security-Policy header (default-src 'self') as a second layer so injected inline scripts are blocked by the browser even if one is missed in output encoding.

Production risk: A stored XSS payload runs in the browser session of every user who views the page, including administrators. In production this enables session-cookie theft, forced actions on behalf of the victim, credential-harvesting overlays, or full account takeover — rated High under OWASP A03:2021.

3. Cross-Site Request Forgery (CSRF)

Objective

Show that DVWA's "change admin password" action can be triggered by a request forged outside the application — i.e. the server accepts the state-changing GET/POST with no anti-CSRF token and no verification that the request originated from DVWA's own form.

Technique used

Step 1 — crafted URL fired directly in the browser while an authenticated DVWA session (cookie) was active:

```
http://192.168.187.4/dvwa/vulnerabilities/csrf/?
password_new=hacked&password_conf=hacked&Change=Change#
```

Step 2 — the same attack repackaged as a standalone, auto-submitting attacker page (csrf_attack.html) to simulate a victim simply visiting a malicious link while logged into DVWA, with no manual URL entry involved:

Exploit evidence

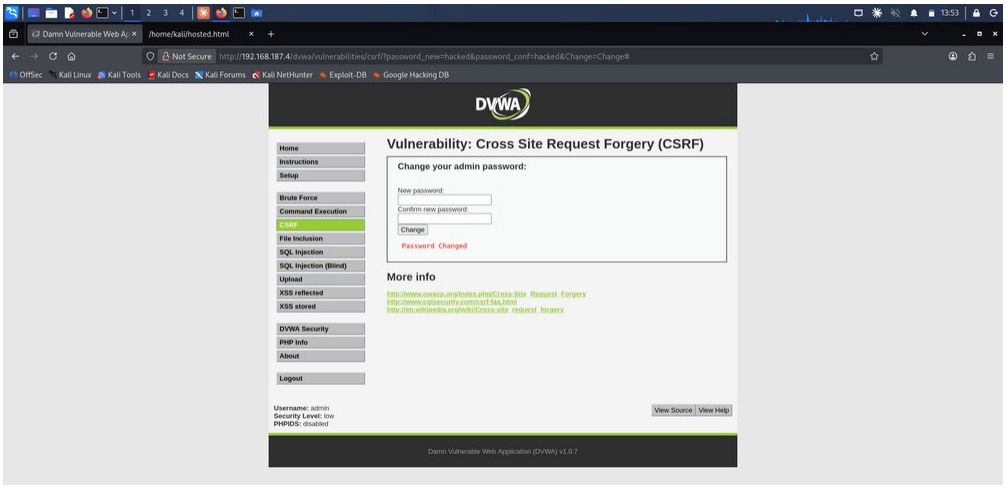


Fig 3.1 — The crafted URL alone changes the admin password ("Password Changed" confirmation), with no CSRF token required.



Fig 3.2 — A locally hosted csrf_attack.html auto-submits the same forged request the instant it loads, exactly as a malicious link embedded in an email or forum post would behave against a logged-in victim.

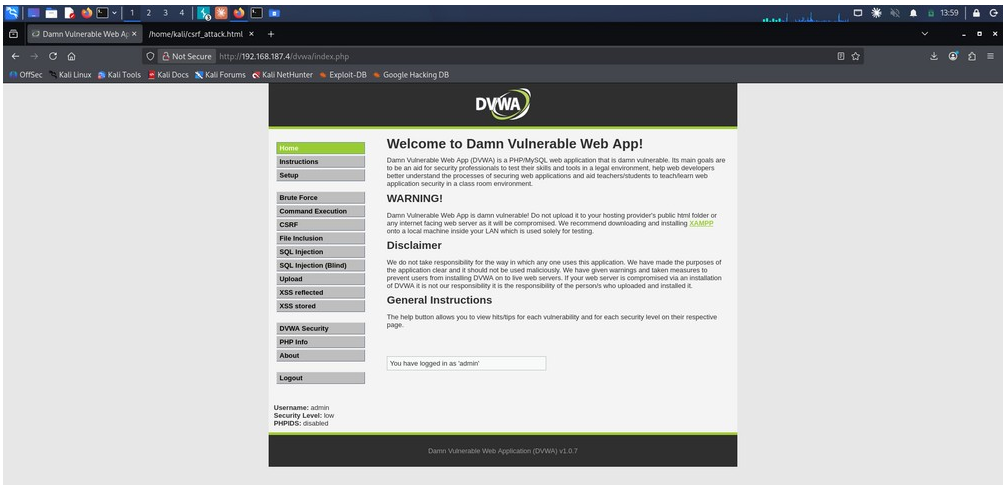


Fig 3.3 — The victim's session is confirmed still authenticated as 'admin' immediately after the forced password change, demonstrating the attack completed silently in the background.

Recommended fix: Issue a unique, unpredictable anti-CSRF token per session (or per form) and require it on every state-changing request; reject requests where the token is missing or mismatched. Also set the session cookie with SameSite=Strict/Lax to stop the browser from attaching it to cross-site requests at all, and re-authenticate (password confirmation) before sensitive actions like a password change.

Production risk: In production, an attacker only needs a logged-in victim to click a link or load an image/page — no credential theft is required. This can be used to change account passwords, transfer funds, alter email addresses for account recovery, or perform any other authenticated action on the victim's behalf. OWASP A01:2021 – Broken Access Control.

4. Local File Inclusion (LFI)

Objective

Determine whether the page= parameter on DVWA's File Inclusion module can be manipulated to read arbitrary files from the underlying server filesystem outside the intended includes directory.

Payloads used

```
http://192.168.187.4/dvwa/vulnerabilities/fi/?page=file:///home/kali/hosted.html/etc/passwd   (failed – malformed path, 404)

http://192.168.187.4/dvwa/vulnerabilities/fi/?page=/etc/passwd   (succeeded)
```

Exploit evidence

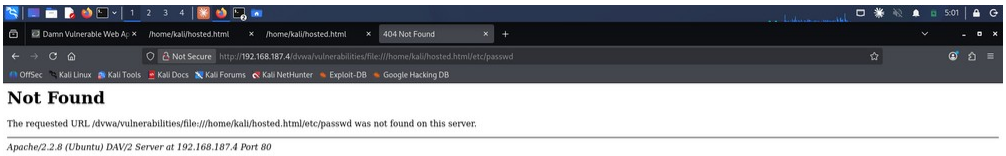


Fig 4.1 — An initial attempt using a file:// wrapper with a malformed path returns a 404, confirming the module does not resolve that exact string.

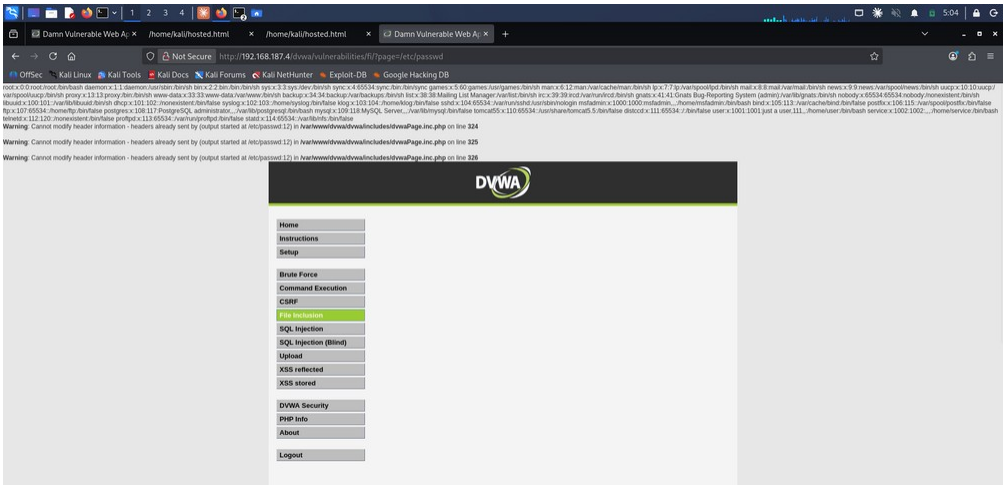


Fig 4.2 — Supplying a direct path (?page=/etc/passwd) returns the full contents of /etc/passwd inline in the page, including every system account and shell assignment — confirming unrestricted LFI.

Recommended fix: Never pass user input directly into include()/require(). Use a whitelist/allow-list that maps short, fixed keywords (e.g. ?page=about) to a hard-coded set of internal filenames, reject any input containing path separators or null bytes, disable allow_url_include and allow_url_fopen in php.ini to also close off Remote File Inclusion, and run the web process with a filesystem user that cannot read sensitive files such as /etc/passwd.

Production risk: LFI in production discloses configuration files, source code, credentials, and SSH keys, and if an attacker can get PHP code onto the disk (upload, log file, session file) LFI escalates directly to remote code execution. OWASP A03:2021 – Injection / A01:2021 – Broken Access Control.

5. Burp Suite Advanced — Request Tampering & Intruder Fuzzing

Objective

Use Burp Suite's intercepting proxy to inspect and modify a live login request in transit, then use Intruder to automate password fuzzing against the DVWA login form.

5.1 — Intercept & modify the login request

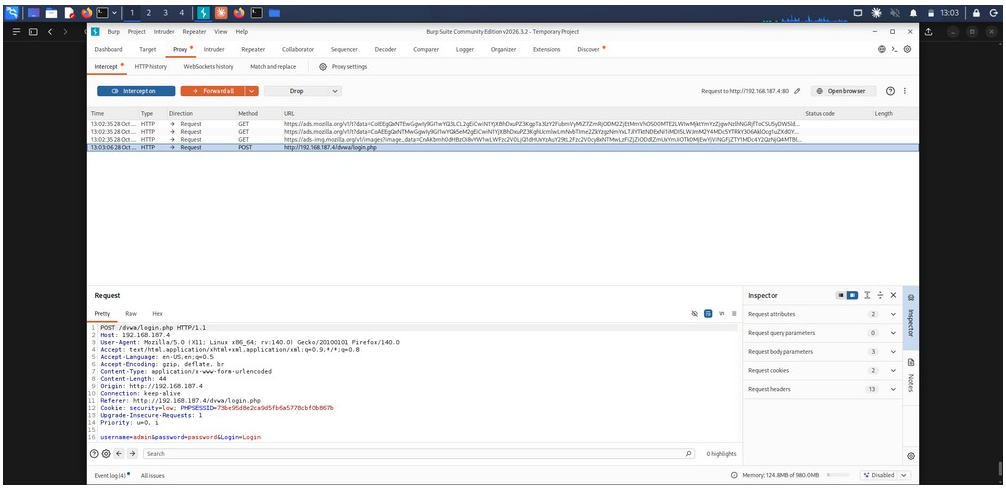


Fig 5.1 — Burp Proxy intercepts the POST /dvwa/login.php request before it reaches the server, exposing the raw username/password/session cookie fields.

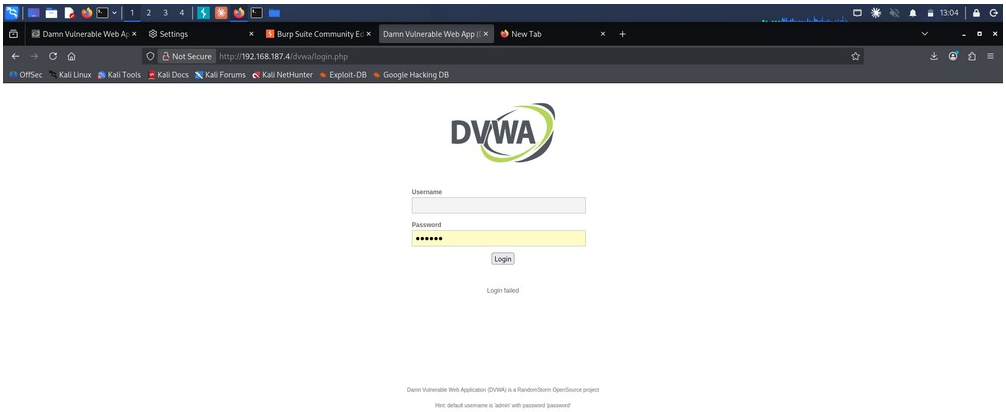


Fig 5.2 — The intercepted request is edited in-flight (username changed to admin123) and forwarded; the server correctly rejects it ("Login failed"), confirming the proxy can tamper with requests the client-side form would never allow.

5.2 — Intruder sniper attack (password fuzzing)

A sniper attack was configured against the password parameter of the login POST, using a short candidate wordlist (12345, letmein, qwerty, password):

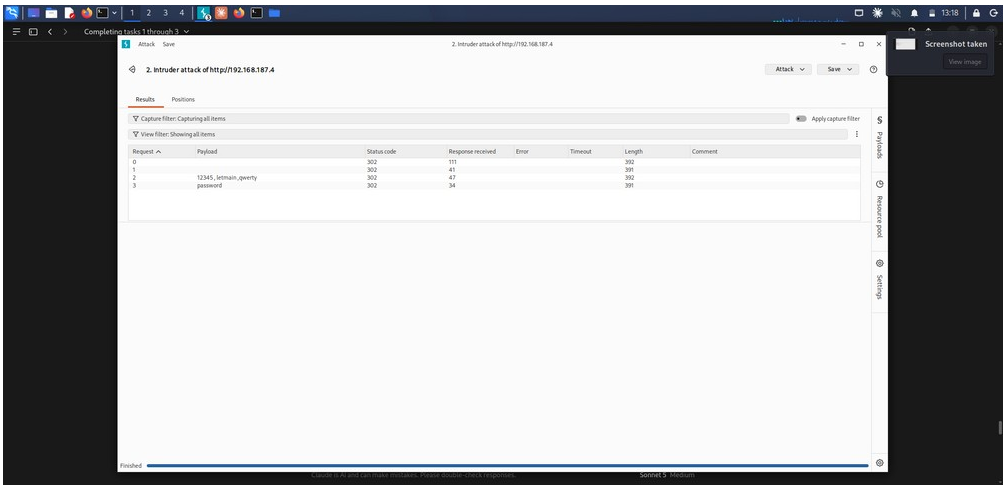


Fig 5.3 — Intruder results: three candidate passwords return a 302 with response length 392 (failed-login pattern), while the correct default credential "password" returns a distinguishable response length of 391 — the classic "different response size = valid guess" signal used to automate credential discovery.

Recommended fix: Enforce account lockout / exponential back-off after repeated failed logins from the same source, add CAPTCHA after a small number of failures, use strong non-default credentials, and keep login failure/success responses identical in size and timing so automated tools cannot distinguish valid from invalid guesses by response length alone.

Production risk: An intercepting proxy demonstrates that client-side validation is not a security boundary — anything the browser would send can be rewritten before it reaches the server. Combined with no rate-limiting, Intruder-style automation turns a login form into a fully automated credential-stuffing / brute-force endpoint. OWASP A07:2021 – Identification and Authentication Failures.

6. Web Security Headers — Before / After Fix

Objective

Audit the HTTP response headers returned by the DVWA/Apache stack for missing browser-security protections, apply the Apache configuration fix, and verify the headers are present afterward — i.e. a genuine before/after mitigation pair, captured with curl -I rather than a third-party grading site, since the lab target sits on a private, non-internet-reachable network.

Before — missing protections

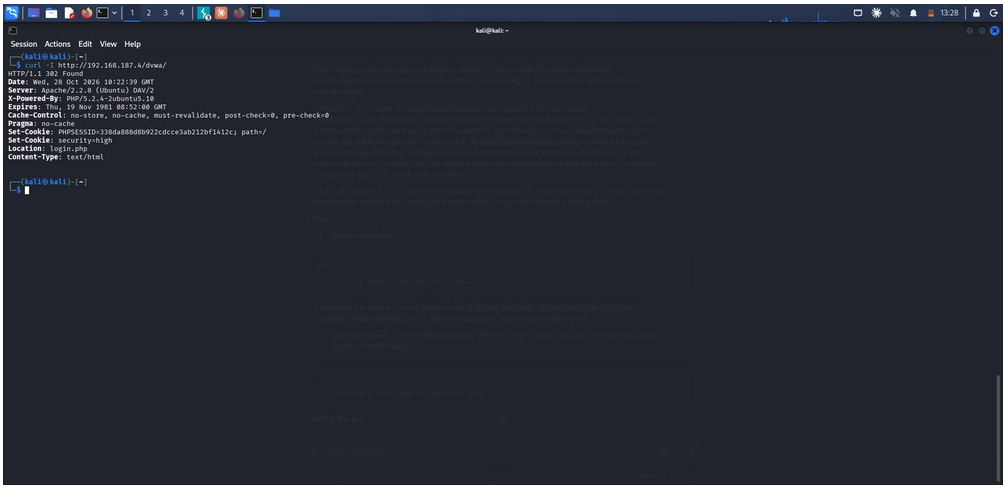


Fig 6.1 — curl -I http://192.168.187.4/dvwa/ before the fix: no X-Frame-Options, no X-Content-Type-Options, and no Content-Security-Policy header is returned at all.

Fix applied

Added to the Apache virtual host configuration (/etc/apache2/apache2.conf):

```
Header set X-Frame-Options "DENY"
Header set X-Content-Type-Options "nosniff"
Header set Content-Security-Policy "default-src 'self'"
```

followed by sudo /etc/init.d/apache2 restart (or systemctl restart apache2).

After — protections present

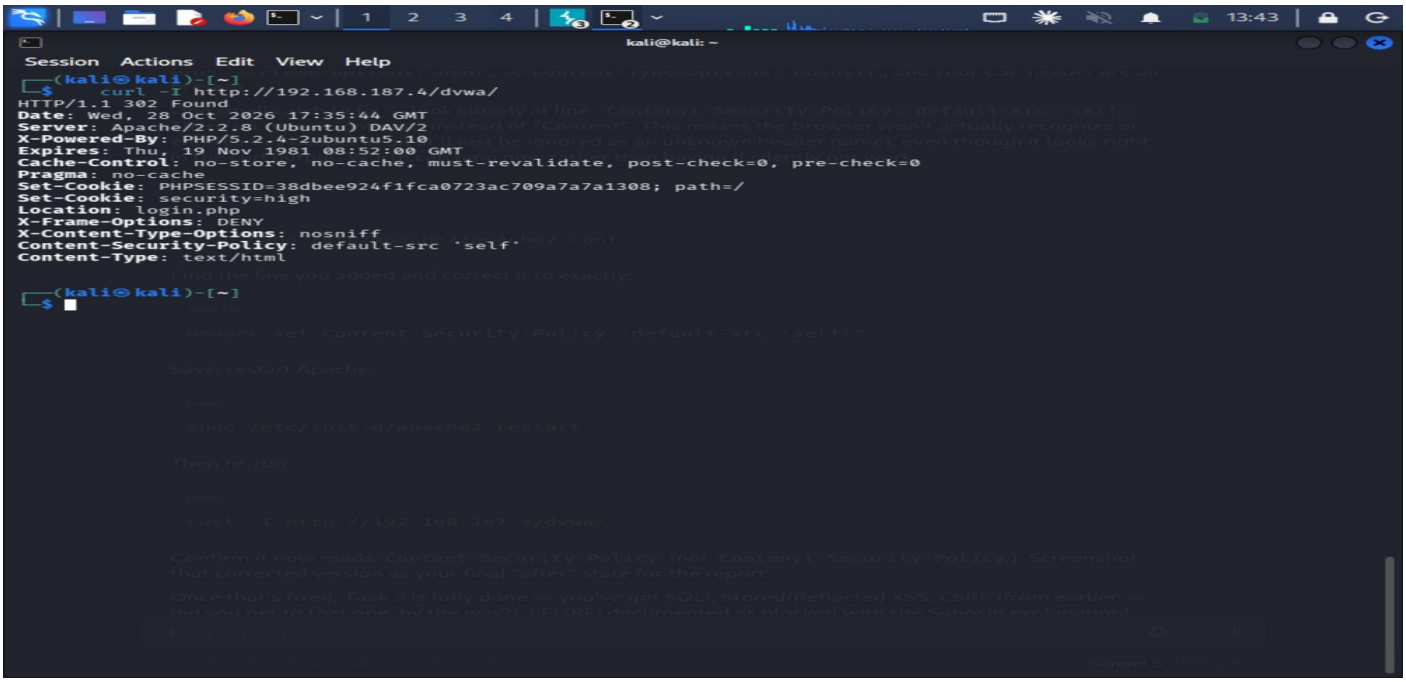


Fig 6.2 — curl -I against the same endpoint after the fix now returns X-Frame-Options: DENY, X-Content-Type-Options: nosniff, and Content-Security-Policy: default-src 'self' on every response.

Recommended fix: Already applied and verified above. As a follow-up, also add Strict-Transport-Security once TLS is enabled, and tighten the CSP beyond default-src 'self' (script-src, style-src, frame-ancestors) once the application's actual resource origins are catalogued.

Production risk: Missing X-Frame-Options/CSP frame-ancestors enables click-jacking (overlaying the real site in a hidden iframe to hijack clicks); missing X-Content-Type-Options allows MIME-sniffing attacks that can turn an uploaded file into executable script; a missing/weak CSP removes the browser's last line of defence against the stored-XSS payload demonstrated in Section 2. OWASP A05:2021 – Security Misconfiguration.

Summary Table

Vulnerability	Severity	Exploited?	Fix verified in this report?
SQL Injection (UNION-based)	Critical	Yes	Recommended — not re-tested at high security
Stored XSS	High	Yes	Recommended — CSP added in Sec. 6 mitigates
CSRF	High	Yes	Recommended — token fix not re-tested

Local File Inclusion	High	Yes	Recommended — not re-tested
Weak auth / no rate-limiting	Medium	Yes (Intruder)	Recommended — not re-tested
Missing security headers	Medium	Yes	Yes — before/after verified

Notes & Disclaimer

All activity described in this report was carried out against DVWA (a deliberately vulnerable training application) and its paired Metasploitable2 host, confined to an isolated, host-only virtual lab network with no route to the internet or to any production system. No real user data, third-party systems, or production credentials were involved at any point.